

What Bug Report Information Helps Agents Fix UI-Centric Bugs? An Empirical Study of Android UI Bug Repair

ANTU SAHA, William & Mary, USA

SPARSH PANDEY, University of Central Florida, USA

ANDRIAN MARCUS, George Mason University, USA

KEVIN MORAN, University of Central Florida, USA

OSCAR CHAPARRO, William & Mary, USA

AI coding agents are increasingly used to fix bugs from issue reports, making the bug report the agent's task specification. Recent studies found that agents benefit most from code-level information, such as fix locations, while traditional report components, such as the steps to reproduce, contribute little. However, these studies focus on developer-written issues for libraries and frameworks. UI-centric bugs present a different setting: their failures are experienced through the GUI, while end-user reports typically describe what users see and do rather than where the fault lies in the code. It is therefore unclear which report information helps agents repair UI-centric bugs.

In this paper, we present the first controlled empirical study of how bug report information affects AI coding agents repairing UI-centric Android bugs. We extend 50 real bugs from 23 open-source Android apps with structured reports and GUI information for their causing and manifesting screens, including screen and component descriptions, screenshots, and UI hierarchies. We derive 17 report configurations that remove traditional report components, add GUI information, or provide code localization. We run  CODEX,  CLAUDE, and  QWEN three times on every bug and configuration, resulting in 7,650 repair attempts. We analyze repair success with mixed-effects logistic regression and agent trajectories to measure repair behavior and effort.

Our results show that report information matters, but its effects depend on the information and the agent. S2Rs have positive estimated effects for all three agents, while the OB is particularly important for  CODEX and the expected behavior adds little. Original user reports outperform structured reports, while UI hierarchies have the strongest positive estimated effect among GUI information. Code localization substantially increases the estimated odds of success for  CLAUDE, but not consistently across agents. Report information also changes repair effort and behavior, sometimes substantially, even when its effect on final success is limited. These findings show that useful report information for UI bug repair differs from what prior work found for library and framework bugs and that no single report configuration benefits all coding agents.

ACM Reference Format:

Antu Saha, Sparsh Pandey, Andrian Marcus, Kevin Moran, and Oscar Chaparro. 2027. What Bug Report Information Helps Agents Fix UI-Centric Bugs? An Empirical Study of Android UI Bug Repair. *Proc. ACM Softw. Eng.* 1, FSE, Article 1 (July 2027), 22 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

Authors' Contact Information: Antu Saha, William & Mary, Williamsburg, Virginia, USA, asaha02@wm.edu; Sparsh Pandey, University of Central Florida, Orlando, Florida, USA, sparsh.pandey@ucf.edu; Andrian Marcus, George Mason University, Fairfax, Virginia, USA, amarcus7@gmu.edu; Kevin Moran, University of Central Florida, Orlando, Florida, USA, kpmoran@ucf.edu; Oscar Chaparro, William & Mary, Williamsburg, Virginia, USA, oscarch@wm.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2994-970X/2027/7-ART1

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Software development is rapidly moving toward agentic workflows [30], in which AI coding agents receive an issue, explore the repository, modify the code, and propose a patch for developers to review [10, 58, 64, 66]. In these workflows, the bug report is no longer documentation for human maintainers only: it becomes the *task specification* for the agent. The agent relies on the report to determine what behavior to reproduce, what behavior was expected, where to look in the codebase, and what code changes to make [15, 16, 37, 43]. The information in the report can therefore impact both whether an agent fixes a bug and how much work it takes to do so [11, 27, 63].

The role of the bug report is particularly important for UI-centric software such as mobile apps. Most mobile app bugs are experienced through the GUI: an app crashes after a tap, a screen shows an incorrect value, or a dialog does not behave as expected [24]. Except for crashes, such failures rarely provide a stack trace or failing unit test that directly points to the cause [23, 31]. Moreover, their reports are often written by end users, who describe what they see and do on the screen rather than what happens in the source code [12, 52, 59]. Yet the bug cause may lie in event handlers, GUI resources, or application state spread across the codebase [34]. Coding agents must therefore turn a user-visible symptom into a successful code change from information that may contain little or no direct reference to the code [34].

Prior work has studied what makes bug reports useful for developers [8, 68]. Three components are particularly important: the *Observed Behavior* (OB), which describes the faulty app behavior; the *Expected Behavior* (EB), which describes the intended behavior; and the *Steps to Reproduce* (S2Rs), which describe the actions to trigger the bug [13, 68]. More recently, researchers have asked what information coding agents need. Studies based on the SWE-bench benchmark found that agents benefit most from operational or code-level information, such as fix code locations and reproduction scripts, while natural language information such as S2Rs contributes relatively little [11, 27]. However, SWE-bench primarily contains developer-written issues for libraries and frameworks. Such reports may include stack traces, code snippets, and hints about where the bug lies. These findings may therefore not transfer to UI bugs reported by end users.

UI-centric bugs also provide another source of potentially useful information: the GUI itself. Information about the UI screens involved in a bug can include screen and component names, screenshots, and UI hierarchies that expose component text, types, and resource IDs. Prior work has used GUI information for tasks such as bug localization, reproduction, and report generation [33, 47, 48]. It is unclear, however, whether coding agents can use these forms of information to repair UI bugs, which representations are useful, and whether their value lies primarily in increasing repair success or in helping agents work more efficiently.

In this paper, we present the first controlled empirical study of how bug report information affects AI coding agents when repairing UI-centric Android bugs. We study three types of information: traditional report components (OB, EB, and S2Rs), bug-related GUI information (e.g., buggy screen descriptions and screenshots), and code localization (e.g., buggy code files and methods).

Our study uses 50 UI-centric bugs from 23 open-source Android apps in the DROIDFIXBENCH benchmark [34]. For each bug, we constructed a structured report and reproduced the bug to collect information about the screen where the bug triggering interaction occurs (the *causing screen*) and the screen where the faulty behavior becomes visible (the *manifesting screen*), i.e., descriptions of the screen, activity, and bug-related components, plus screenshots and UI hierarchies. Using these artifacts, we constructed 17 report configurations, including a controlled *reference* report containing a title, OB, EB, and S2Rs, and variants that remove traditional report components, add different forms of GUI information, or provide the files and methods changed by the developer fix to the bug. We instructed three state-of-the-art agents, namely  CODEX,  CLAUDE, and  QWEN,

to fix every bug under every configuration, repeating each attempt three times, for a total of 7,650 repair attempts. We analyzed repair success using mixed-effects logistic regression and the agents' trajectories to determine how report information affects both repair outcomes and agent behavior.

Our results show that report information can affect repair success, but the actual bug and agent have a much larger influence on outcomes. Bug identity accounts for 51–88% of the variation in repair outcomes, compared with only 3–4% for report configuration. Still, specific report information changes the odds of success: S2Rs have positive estimated effects for all three agents, with a combined OR of 1.63, while the OB has a particularly strong effect for  CODEX. GUI and code localization information also change repair outcomes, although their effects are less consistent across agents. The clearest effects of report information are often on repair effort: code localization information lets  CODEX and  CLAUDE reach developer-modified files about four times faster and reduces their median repair cost by 32% and 42%, respectively, while GUI information reduces effort for some agents and helps  QWEN localize the relevant code better. Yet finding the right code does not guarantee successful repairs: 48% of failed attempts already modify the same method changed by the developer, and 74% of failed tests fail when checking the expected behavior. We also find substantial variation across agents and runs: report configurations affect the three agents differently, and run-to-run variation is particularly high for  QWEN.

These findings suggest that there is no single “best” bug report for coding agents fixing UI-centric Android bugs. Traditional report information remains useful, especially for describing the faulty behavior to reproduce, but richer reports do not consistently lead to more successful repairs. Instead, report information often helps agents work more efficiently, while many failures occur after the agent has already found the relevant code. These findings suggest that successful UI bug repair involves challenges beyond code localization: agents often reach the relevant code but still fail to produce the expected behavior.

In summary, this paper makes the following contributions:

- A controlled study of how bug report components, GUI information, and code localization affect AI coding agents fixing UI-centric bugs, based on 7,650 repair attempts on 50 real Android bugs using three state-of-the-art coding agents:  CODEX,  CLAUDE, and  QWEN.
- We show that the bug and agent influence repair success more than the report, but report information still matters. S2Rs improve success across agents, while the OB strongly helps  CODEX.
- We show that report information often affects how efficiently agents repair bugs rather than whether they succeed. Our trajectory analysis also shows that many failures occur after agents have already found the faulty code.
- We extend DROIDFIXBENCH with structured reports and GUI information for all 50 bugs, including screen, activity, and component descriptions, screenshots, and UI hierarchies for bug causing and manifesting screens.
- We provide a replication package with the dataset, report configurations, agent trajectories, generated patches, experiment infrastructure, and analysis scripts [3].

2 Background and Motivation

2.1 Bug Reports for UI-centric Apps

For Android bug reports, Saha *et al.* [47] proposed a quality model that defines what the *observed behavior* (OB), the *expected behavior* (EB), and the *steps to reproduce* (S2Rs) should contain. A high-quality S2R is an atomic GUI interaction, *i.e.*, a single action (*e.g.*, a tap) on a single GUI component (*e.g.*, the “OK” button). A high-quality OB describes the *buggy behavior*, the *triggering GUI interaction*, and the *buggy screen reference* (*i.e.*, the screen where the bug is observed), and a

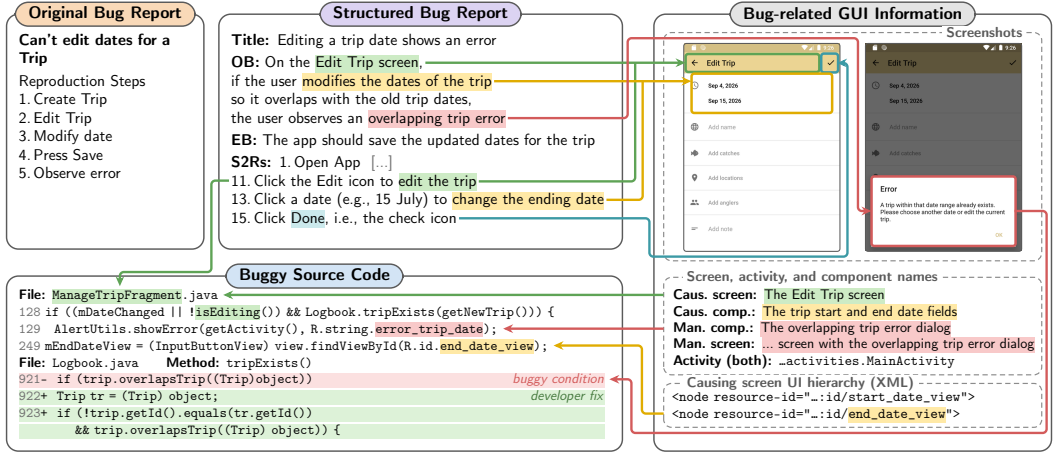


Fig. 1. Bug #347 of Anglers' Log: the original and structured reports, the bug-related GUI information, and the buggy code. The arrows link the report and GUI information to the code.

high-quality EB describes the *intended behavior*. We call a report whose components follow this model a *structured bug report* (see Figure 1).

2.2 GUI Information in Bug Reports

Since the faulty behavior of a UI bug is visible on the screen, information about the app's GUI (i.e., GUI information) can help in understanding and localizing the fault at the code level. Two particular types of screens are most relevant to a bug. The *causing screen* is the screen on which the user performs the triggering GUI interaction (e.g., the screen with the button that the user taps), and the *manifesting screen* is the screen on which the faulty behavior becomes visible (e.g., an error dialog or a screen with wrong content). The two screens can be the same, for instance, when a button does not update the screen that contains it, or they can be different, such as when a toggle switch set on a prior screen causes a crash on another.

UI screens can be represented in a variety of ways. In our study we explore three representations. First, the *screen, activity, and component names* describe the screen, the Android activity that displays it, and potential GUI component(s) involved in the bug (e.g., "the trip start and end date fields"). Screen and component names can be expressed in natural language based on the labels shown on the screen or based on its implemented functionality; activity names can be obtained from screen metadata. Second, a *screenshot* shows the visual appearance of the screen in pixels. Third, the *UI hierarchy* (i.e., an XML file) represents the structure of the screen, including the type, text, and resource ID of each component. Resource IDs are particularly relevant for repair, since developers use the same IDs in the source code to access the components (e.g., `R.id.end_date_view`). Screenshots and UI hierarchies can be obtained via the UIAutomator tool [2]. LLM-based agents that operate apps also commonly read the UI hierarchy, which can be compacted to reduce the tokens it costs [45]. Prior work has shown the benefit of UI information in bug report management tasks for UI-centric mobile apps. It has shown to be useful in improving localizing the buggy screens of a report [48], localizing the buggy code [33], replaying visual bug reports [17], and generating high-quality reports [47, 52].

2.3 Motivating Example

Figure 1 shows bug #347 of Anglers' Log [1], an app for logging fishing trips. The original report states that the user "can't edit dates for a Trip" and lists five high-level steps that end with "Observe error". For an agent, this report gives little to work with: it does not say which error appears, on which screen, or under which conditions. The structured report is more precise. Its OB states that, on the *Edit Trip* screen, modifying the trip dates so that they overlap with the old dates produces an *overlapping trip error*, and its S2Rs describe the exact interactions (e.g., "Click Done, i.e., the check icon"). Still, both reports describe the bug in terms of the screen, while the agent has to find and fix code. The bug lies in `Logbook.tripExists()`, which checks whether a trip overlaps with any existing trip, including the trip that is being edited. Editing the dates of a trip therefore makes it overlap with itself, and the developer fixed the bug by skipping the trip's own ID.

The GUI information of the bug contains clues that connect the report to this code. For example, the name of the causing screen ("Edit Trip") partially matches the `ManageTripFragment` class and its `isEditing()` check. The UI hierarchy of the causing screen contains the resource ID `end_date_view`, which the same class uses to access the date field that the user modifies. The manifesting screen shows the error dialog, whose message comes from the string `error_trip_date`, which is displayed right after the call to `tripExists()`. An agent with these clues can move from the symptom on the screen to the faulty method, without knowing the code location in advance. With this information, the agent can analyze the code and determine the correct patch for the bug.

3 Study Design

We aim to understand how bug report information affects AI coding agents fixing UI-centric bugs in Android apps. We examine both repair outcomes and the agents' repair behavior, focusing on three types of information: traditional bug report components, bug-related GUI information, and code localization. With this in mind, we formulate the following research questions (RQs):

- **RQ₁**: How do different bug report components affect the agents' ability to fix UI bugs?
- **RQ₂**: How do different bug report components affect the agents' repair behavior and effort?

RQ₁ examines how different bug report components affect agents' ability to fix UI bugs. This includes traditional report components, such as the OB, EB, and S2Rs, as well as bug-related GUI information and code localization. Recent studies of library and framework bugs found that some report components, such as the S2Rs, contribute little to repair success [11, 27], but it is unclear whether these findings also hold for UI-centric Android bugs. **RQ₂** examines how report components affect the agents' repair behavior and effort, including their code exploration and localization, tool use and cost, and patch validation.

3.1 Dataset Construction

3.1.1 Bug Dataset. We use and extend DROIDFIXBENCH [34], a recent benchmark of real UI-centric Android bugs designed for automated program repair studies. Unlike earlier benchmarks of real Android bugs, which target GUI testing [54], it provides the developer fixes and tests needed to evaluate repairs. It contains 50 bugs from 23 open-source Android apps from different domains (education, entertainment, health, etc.). The dataset provides the artifacts needed to execute and evaluate repair attempts on UI-centric bugs, yet it lacks GUI information about the apps and bugs.

For each bug, the benchmark provides: (1) the original user-submitted bug report; (2) a compilable version of the app immediately before the bug was fixed; (3) the developer patch, including the files, methods, and lines changed by the fix; and (4) two GUI-level test cases. The first test reproduces the reported bug and checks its expected behavior, failing on the buggy version and passing on the

fixed version. The second checks related functionality and passes on both versions. We use both tests to evaluate generated patches (Section 3.4).

The dataset covers the 4 common types of UI-centric Android bugs [24]: 12 crashes, 25 output issues, 9 cosmetic issues, and 4 navigation issues. The developer fixes modify 97 methods in 63 Java files, removing 290 lines and adding 414. Twenty-seven bugs are fixed within one method, while 23 in multiple methods. These characteristics provide variation in both bug type and repair size.

3.1.2 Structured Bug Report Construction. The original user-submitted bug reports vary in content, completeness, and wording, which makes it difficult to isolate the effect of individual types of information. We therefore created a *structured report* for each bug and used it as the basis for the report controlled configurations in Section 3.2. Each structured report contains a title, observed behavior (OB), expected behavior (EB), steps to reproduce (S2Rs), and additional information.

We constructed the OB, EB, and S2Rs following the quality model of Saha *et al.* [47]. Twelve DROIDFIXBENCH bugs are included in their curated dataset, so we reused their OB, EB, and S2Rs. We constructed these components for the remaining 38 bugs as follows.

S2Rs. We analyzed each bug reproduction test to identify its sequence of actions, then performed those actions on Pixel 2 emulator running Android 9. This was necessary because the tests often identify GUI components through code-level selectors rather than the information visible to an end user. We expressed each step using the template from prior work [47], “[action] [GUI component]” (e.g., “Tap the ‘More options’ button”).

OB and EB. We obtained the buggy and intended behaviors from the original report and the bug triggering interaction and buggy screen from the reproduced S2Rs. When a screen had a title or label indicating its function, we used it to name the screen; otherwise, we assigned a concise name based on its main function. We then wrote the OB and EB using the templates of Saha *et al.* [47], which are based on discourse patterns found in real bug reports [13]. The OB follows the form “On [buggy screen name], if the user [triggering GUI interaction], [buggy app behavior]” (e.g., “On the confirmation dialog, if the user taps the OK button, the app crashes”), while the EB states the intended behavior using “[subject] should/shouldn’t [intended app behavior]” (e.g., “the app should save the note instead of crashing”).

Title and Additional Information. For all 50 bugs, we wrote a concise title and relevant additional bug information, such as device or environment details. The dataset of Saha *et al.* [47] does not provide these two components, so we constructed them ourselves. We rewrote the original title or additional information when it was incomplete, unclear, or difficult to interpret, using the original report and the structured OB, EB, and S2Rs as references.

3.1.3 Bug-Related GUI Information. For each bug, we collected GUI information for the *causing screen*, where the bug triggering interaction occurs, and the *manifesting screen*, where the faulty behavior becomes visible. For each screen, we collected the screen name, activity name, bug-related component information, as well as a screenshot, and its UI hierarchy.

We reproduced all 50 bugs on a Pixel 2 emulator running Android 9. 🦾 CLAUDE [5] executed the structured S2Rs, and we video recorded each reproduction. We manually inspected the recordings and confirmed that all bugs were reproduced as described in the structured reports.

During reproduction, we captured the screenshot, UI hierarchy, and activity name of both screens using the Android’s UIAutomator tool [2]. The UI hierarchy records the structure and attributes of the GUI components on the screen (labels, types, and other metadata). We wrote the screen and component descriptions by inspecting the reproduction video, captured screens, and structured report. These descriptions summarize the functionality visible to the user, typically using text shown on the screen (e.g., “The edit trip dialog”).

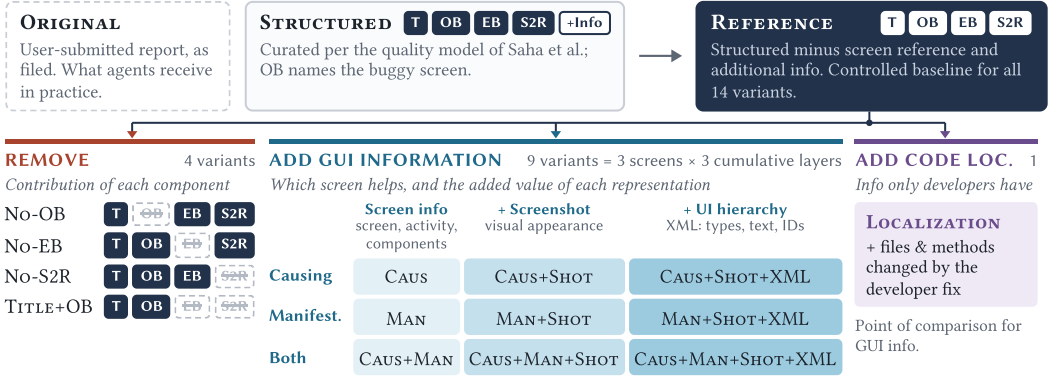


Fig. 2. The 17 bug report configurations. REFERENCE is derived from STRUCTURED, and the 14 controlled variants are derived from REFERENCE by removing traditional components, adding GUI information for the causing screen, the manifesting screen, or both (each layer adds to the previous one), or adding the code locations changed by the developer fix.

3.1.4 Validation. Two researchers were involved in the manual construction and validation of the study artifacts. For each bug, one researcher constructed the structured report, reproduced the bug, collected its GUI information, and wrote the screen and component descriptions. A second researcher validated these artifacts against the original report, reproduction test, and observed app behavior. The researchers discussed disagreements until reaching consensus. The second researcher agreed with 90.6% of the constructed elements (794 of 876). Agreement was 94.0% for the titles, 89.2% for the OB, 65.8% for the EB, 90.4% for the S2Rs, 96.0% for the additional information, and 93.0% and 95.0% for the descriptions of the causing and manifesting screens and their components. The most common disagreements involved interpreting an information element or the buggy behavior.

3.2 Bug Report Configurations

We constructed 17 report configurations for each bug. A *configuration* is a complete report containing a specific combination of components. The configurations let us vary the report information while keeping certain components fixed.

The study includes three reports used as reference points, four traditional-component variants, nine GUI-information variants, and one code-localization variant. The REFERENCE configuration is derived from the structured report, and all controlled variants are derived from REFERENCE. This design keeps the underlying bug and report wording fixed while changing the information given to the agent. Figure 2 illustrates the configurations.

3.2.1 Reference and Comparison Reports. ORIGINAL is the original user-submitted bug report on the apps' issue trackers. STRUCTURED is the structured report described in Section 3.1.2. REFERENCE is the controlled baseline for the experimental comparisons.

We derive REFERENCE from STRUCTURED by removing the buggy screen reference from the OB and the additional information. We remove these components because the screen reference overlaps with the configurations that include GUI information, while additional information varies across bugs. Thus, REFERENCE contains the title, OB, EB, and S2Rs, but none of the GUI or additional information. We compare ORIGINAL and STRUCTURED with REFERENCE to measure whether the controlled reference behaves differently from the other two reports.

3.2.2 Traditional-Component Variants. NO-OB, NO-EB, and NO-S2R remove the OB, EB, and S2Rs, respectively, from REFERENCE. Each configuration isolates the effect of removing one traditional component. TITLE+OB retains only the title and OB (*i.e.*, without the EB or S2Rs), providing a minimal report describing the bug. These configurations test whether the traditional components help agents repair UI bugs.

3.2.3 GUI-Information Variants. The GUI-information configurations add the GUI-related artifacts collected in Section 3.1.3 to REFERENCE. They vary by which screen is included and by the information provided for that screen.

We provide information about the causing screen (CAUS), manifesting screen (MAN), or both (CAUS+MAN). This lets us distinguish information about where the triggering interaction occurs from information about where the faulty behavior becomes visible. For each included screen, the first level of GUI information provides its screen name, activity name, and bug-related component information (the GUI component interacted with or where the bug manifests). For brevity, we refer to these three fields together as *screen information*. We then add the screenshot (*e.g.*, CAUS+SHOT) and the UI hierarchy (*e.g.*, CAUS+SHOT+XML). This layered design measures the added effect of screenshots when screen information is already present and of UI hierarchies when both screen information and screenshots are present.

3.2.4 Code Localization. LOCALIZATION adds the files and methods modified by the developer patch to REFERENCE (Section 3.1.1). These elements represent the code affected by the bug. Prior work found code localization particularly useful for coding agents when repairing library/framework bugs [11, 27]. We include this configuration to measure the effect of code-level localization information on agentic UI-centric bug repair.

3.3 Coding Agents and Execution Setup

3.3.1 Coding Agents. We evaluate  CODEX [42] (v. 0.138.0),  CLAUDE [5] (v. 2.1.181), and  QWEN [44] (v. 0.24.2). Each is a command-line coding agent that can inspect a repository, execute commands, modify files, and validate changes.  CODEX uses GPT-5.6-Sol,  CLAUDE uses Opus 5, and  QWEN uses Qwen3.6-35B-A3B. This selection provides state-of-the-art agents from different providers and includes both proprietary and open-weight models.

We used each agent's default tools and settings except for permission handling. We configured all agents to operate autonomously without requesting approval:  CODEX with full access and no approvals,  CLAUDE in bypass-permissions mode, and  QWEN in YOLO mode.

3.3.2 Prompt. We used the same prompt template for all agents and configurations. The template assigns the agent the role of an experienced software engineer fixing an Android bug, presents the bug report, and instructs it to understand the bug, locate the relevant code, and produce a patch that builds, fixes the reported behavior, and preserves existing functionality. The prompt also provides a command for building the apps: `./gradlew clean assembleDebug`. The full template is available in our replication package [3].

The bug report is the only part of the prompt that varies across configurations. When included, GUI and code localization information appears in a separate *Location* section. For configurations with separate artifacts, such as screenshots or UI hierarchies, the report provides their file paths. These artifacts are placed in the app's project directory, which contains the buggy version of the app and only the artifacts available for that configuration. The agents have access to this project directory but not to the benchmark tests used for evaluation.

3.3.3 Execution Environment. Before the experiments, we verified that every bug could be reproduced on the emulator using its reproduction test. Each repair attempt then ran in a fresh copy

of the buggy project, so no attempt could observe changes from another run. Agents could read and modify files, execute shell commands, build the app, and interact with the emulator, including installing and exercising the app. We disabled network access to prevent retrieval of the developer fix from external sources such as the issue tracker or commit history. We imposed no step, turn, or time limit, so each agent ran until it considered the task complete.

3.3.4 Repair Attempts. A *repair attempt* is one independent agent session for one bug under one report configuration. The agent receives one opportunity to solve the bug within that session and receives no feedback from our evaluation. To account for LLM non-determinism, we repeated every configuration and agent combination three times. The study therefore contains 7,650 repair attempts: $50 \text{ bugs} \times 17 \text{ configurations} \times 3 \text{ agents} \times 3 \text{ runs}$.

3.4 Evaluation Metric

We evaluate each attempt using the *plausible patch* metric commonly used in APR studies [29, 61]. A patch is plausible if the modified app builds successfully and passes the tests for the bug.

After each attempt, we extract the agent’s patch by comparing its modified project/repository with a clean copy of the buggy version. We remove any test files created by the agent so that agent-written tests do not affect evaluation. We then apply the remaining patch to a clean copy of the buggy version of the project, build the app, and run both GUI-level tests described in Section 3.1.1. The resulting binary outcome is the dependent variable in the statistical analysis (see Section 3.5).

3.5 Statistical Analysis

To answer RQ₁, we use mixed-effects logistic regression, with patch plausibility as the dependent variable. We include random intercepts for bugs and apps to account for repeated attempts on the same bugs and for bugs from the same app. We report effects as odds ratios (ORs) with 95% confidence intervals.

We fit two complementary models. The *feature model* estimates the effect of individual information components contained in a report. It estimates how each component changes the odds of producing a plausible patch relative to bug reports with no such component. We use binary predictors that indicate whether the report includes the OB, EB, S2Rs, causing screen information, manifesting screen information, screenshots, UI hierarchies, and code localization data. We fit this model on REFERENCE and its 14 controlled variants, excluding ORIGINAL and STRUCTURED because they differ from REFERENCE in information not represented by these predictors. Because screenshots are always added on top of screen information, and UI hierarchies on top of screenshots, their effects represent the added value of these forms of information. We also test $EB \times S2Rs$ and $causing\text{-}screen \times manifesting\text{-}screen$ interactions.

The *configuration model* treats the 17 report configurations as a categorical predictor, with REFERENCE as the baseline. It estimates how each complete report configuration changes the odds of producing a plausible patch relative to REFERENCE.

We fit both models separately for each agent and jointly across all agents. The combined models account for differences among agents and test whether report configuration effects differ across them. We fit the models with R’s package lme4 [7], report Wald 95% confidence intervals, and adjust p -values for multiple comparisons using the Holm method [20]. We consider adjusted $p < 0.05$ statistically significant.

Consistency of the Outcomes. As the agents are nondeterministic, we also examine how consistent the outcomes are across runs, agents, configurations, and bugs of different complexity, by inspecting the outcome of every attempt. To examine the role of bug complexity, we group the bugs into three tiers by the hunk divergence of the developer fix [41] (i.e., how much scattered are

blocks of changed coded), computed without the hunks unrelated to the fix, which combines the lexical, syntactic, and file distance between its hunks, using Fisher–Jenks natural breaks [19]: 19 easy, 24 medium, and 7 hard bugs.

3.6 Trajectory Analysis

To answer RQ₂, we parse and analyze the trajectories of all 7,650 repair attempts. A trajectory records the actions performed by an agent during a repair session and their outputs. Because the three agents use different logging formats and tools, we implemented one parser per agent to extract actions into a common representation, including the invoked tool, its arguments and output, and whether it returned an error. We split chained commands, remove polling actions, and classify the remaining actions into categories such as search, read, edit, build, test, diff, and setup using rules based on tool names and commands. The parsers and complete classification rules, based on text matching, are available in our replication package [3].

From each trajectory, we compute metrics that represent agent *effort* in repairing a bug, including the number of tool actions, token usage, cost, and the duration of a repair attempt.

Finally, we analyze unsuccessful repair attempts in two stages. First, we label each failed attempt with *where* it failed, using the first rule that applies: the run is incomplete (*i.e.*, the log has no final record of a finished run); the patch is empty; the patch changes no app code (*e.g.*, only tests or build files); the patch edits only files that the developers did not change; it edits a file changed by the developers, but not a method they changed; or it edits a method changed by the developers. We classify test failures separately from the validation logs.

Second, to understand *why* the agents fail, we analyze a sample of 338 of the 2,486 failed attempts (95% confidence level, 5% margin of error), stratified by agent, run, and configuration. LLMs can replace human annotators for some software engineering tasks, especially when the models agree with each other [4]. We therefore use  CODEX,  CLAUDE, and  QWEN as committee of “judges” to independently inspect each sampled attempt and explain the failure using evidence from the report, trajectory, patch, validation log, reproduction test, and developer patch. The judges code each attempt twice. First, each judge describes the root cause in its own words. From these descriptions, an agent (*i.e.*,  CLAUDE) drafts a codebook, which an author reviews and refines into 12 categories grouped into four themes. Second, each judge labels each attempt with one category of the codebook. We assign the label chosen by at least two judges, and we measure the judges’ agreement with the share of attempts with a majority label, pairwise Cohen’s κ [14], and Krippendorff’s α [28].

Use of AI assistance. Separately from our experiments, we used  CLAUDE to help write the scripts for data extraction, statistical analysis, figures, and patch generation and validation.

4 Results

4.1 RQ₁: Effect of Bug Report Components on Repair Success

Table 1 reports the estimated effect of each report component relative to reports without that component (*e.g.*, S2Rs vs no S2Rs), while Table 2 reports the effect of each complete report configuration relative to REFERENCE. We interpret each odds ratio (OR) as an effect-size estimate: an OR of 1 indicates no difference in the odds of a plausible patch, an OR above 1 indicates higher odds, and an OR below 1 indicates lower odds. We report each OR with its 95% confidence interval. An interval that includes 1 indicates that the direction or magnitude of the effect is uncertain, rather than that the component has no effect.

Traditional report components. The S2Rs have the strongest and most consistent positive effect, according to Table 1. Their estimated OR is above one for all three agents, and the combined estimate is 1.63. This is consistent with the results in Table 2. Compared to REFERENCE, removing

Table 1. Feature model: odds ratio (OR) of each piece of report information, with 95% confidence interval and Holm-adjusted p -value, per agent and for all agents, estimated on REFERENCE and its variants. The last two rows test whether two pieces of information interact (likelihood-ratio test). Bold: $p < 0.05$.

	CODEX			CLAUDE			QWEN			All agents		
	OR	95% CI	p	OR	95% CI	p	OR	95% CI	p	OR	95% CI	p
OB	3.38	[1.48, 7.70]	0.030	1.62	[0.58, 4.53]	1.000	0.84	[0.51, 1.37]	1.000	1.27	[0.85, 1.88]	0.968
EB	0.67	[0.34, 1.29]	0.965	0.99	[0.45, 2.19]	1.000	0.99	[0.67, 1.45]	1.000	0.91	[0.67, 1.24]	1.000
S2Rs	2.44	[1.26, 4.73]	0.056	2.11	[0.96, 4.65]	0.438	1.36	[0.92, 2.00]	0.954	1.63	[1.20, 2.22]	0.013
Causing screen	0.94	[0.59, 1.49]	1.000	0.66	[0.38, 1.15]	0.732	1.03	[0.78, 1.35]	1.000	0.94	[0.76, 1.17]	1.000
Manifesting screen	0.61	[0.38, 0.98]	0.245	0.62	[0.36, 1.09]	0.574	0.97	[0.74, 1.27]	1.000	0.82	[0.66, 1.02]	0.430
Screenshot	0.77	[0.45, 1.32]	1.000	1.00	[0.52, 1.89]	1.000	1.12	[0.81, 1.54]	1.000	1.01	[0.78, 1.31]	1.000
UI hierarchy (XML)	1.46	[0.83, 2.58]	0.965	1.20	[0.61, 2.36]	1.000	1.30	[0.93, 1.82]	0.954	1.32	[1.01, 1.73]	0.284
Code localization	1.01	[0.42, 2.41]	1.000	4.22	[1.19, 14.88]	0.202	1.25	[0.76, 2.05]	1.000	1.35	[0.90, 2.01]	0.719
<i>Interactions</i>												
EB $\times$ S2Rs	0.92	[0.25, 3.49]	1.000	0.71	[0.14, 3.50]	1.000	1.72	[0.79, 3.78]	0.177	1.32	[0.71, 2.44]	0.379
Causing $\times$ manifesting	1.38	[0.51, 3.72]	1.000	1.03	[0.31, 3.40]	1.000	0.48	[0.27, 0.85]	0.026	0.67	[0.42, 1.06]	0.176

Table 2. Configuration model: plausible-patch rate of each configuration and its odds ratio (OR) against REFERENCE, with 95% confidence interval and Holm-adjusted p -value, per agent and for all agents. Bold: $p < 0.05$. Rate: percentage of plausible patches among all attempts of the configuration (all runs). Attempts: CODEX 2,550, CLAUDE 2,550, QWEN 2,550.

	CODEX				CLAUDE				QWEN				All agents			
	Rate	OR	95% CI	p	Rate	OR	95% CI	p	Rate	OR	95% CI	p	Rate	OR	95% CI	p
<i>Baselines</i>																
REFERENCE (baseline)	77	1.00	–	–	89	1.00	–	–	35	1.00	–	–	67	1.00	–	–
ORIGINAL	83	4.46	[1.50, 13.30]	0.117	90	1.62	[0.42, 6.32]	1.000	49	2.37	[1.33, 4.20]	0.053	74	2.46	[1.55, 3.92]	0.002
STRUCTURED	75	0.67	[0.25, 1.84]	1.000	87	0.55	[0.16, 1.90]	1.000	38	1.19	[0.67, 2.13]	1.000	66	0.95	[0.60, 1.49]	1.000
<i>Component omission</i>																
No-OB	69	0.26	[0.10, 0.70]	0.117	87	0.66	[0.19, 2.34]	1.000	41	1.48	[0.83, 2.64]	1.000	66	0.90	[0.57, 1.41]	1.000
No-EB	78	1.32	[0.47, 3.68]	1.000	89	1.26	[0.33, 4.80]	1.000	37	1.09	[0.61, 1.95]	1.000	68	1.15	[0.73, 1.81]	1.000
No-S2R	71	0.37	[0.14, 0.99]	0.609	87	0.55	[0.16, 1.90]	1.000	32	0.80	[0.44, 1.43]	1.000	63	0.65	[0.41, 1.02]	0.920
TITLE+OB	73	0.53	[0.19, 1.42]	1.000	86	0.46	[0.14, 1.57]	1.000	37	1.09	[0.61, 1.95]	1.000	65	0.83	[0.52, 1.30]	1.000
<i>GUI augmentation</i>																
CAUS	75	0.77	[0.28, 2.10]	1.000	89	1.00	[0.27, 3.71]	1.000	41	1.48	[0.83, 2.64]	1.000	68	1.21	[0.77, 1.92]	1.000
CAUS+SHOT	74	0.60	[0.22, 1.62]	1.000	86	0.46	[0.14, 1.57]	1.000	43	1.62	[0.91, 2.87]	1.000	68	1.09	[0.69, 1.72]	1.000
CAUS+SHOT+XML	76	0.87	[0.32, 2.41]	1.000	89	1.00	[0.27, 3.70]	1.000	46	2.00	[1.13, 3.55]	0.273	70	1.51	[0.96, 2.40]	0.994
MAN	73	0.47	[0.17, 1.26]	1.000	87	0.66	[0.19, 2.34]	1.000	42	1.55	[0.87, 2.75]	1.000	67	1.06	[0.67, 1.67]	1.000
MAN+SHOT	73	0.53	[0.19, 1.42]	1.000	91	2.11	[0.53, 8.48]	1.000	43	1.62	[0.91, 2.87]	1.000	69	1.28	[0.81, 2.03]	1.000
MAN+SHOT+XML	73	0.47	[0.17, 1.26]	1.000	85	0.34	[0.10, 1.12]	1.000	43	1.62	[0.91, 2.87]	1.000	67	0.97	[0.62, 1.54]	1.000
CAUS+MAN	73	0.53	[0.19, 1.42]	1.000	85	0.34	[0.10, 1.12]	1.000	38	1.19	[0.67, 2.13]	1.000	65	0.83	[0.52, 1.30]	1.000
CAUS+MAN+SHOT	71	0.33	[0.12, 0.88]	0.367	84	0.29	[0.09, 0.96]	0.630	35	1.00	[0.56, 1.79]	1.000	63	0.65	[0.41, 1.02]	0.920
CAUS+MAN+SHOT+XML	75	0.77	[0.28, 2.10]	1.000	89	1.26	[0.33, 4.80]	1.000	44	1.76	[0.99, 3.12]	0.756	70	1.39	[0.88, 2.20]	1.000
<i>Code localization</i>																
LOCALIZATION	76	0.87	[0.32, 2.41]	1.000	93	5.16	[1.18, 22.53]	0.464	42	1.55	[0.87, 2.75]	1.000	70	1.51	[0.96, 2.40]	0.994

the S2Rs (No-S2R) reduces the plausible patch rate from 77% to 71% for CODEX, from 89% to 87% for CLAUDE, and from 35% to 32% for QWEN. The OB also matters, particularly for CODEX and CLAUDE. For CODEX, its estimated OR is 3.38, and removing it reduces CODEX's plausible-patch rate from 77% to 69%. In contrast, the EB has little effect: its estimated ORs are close to or below one for all three agents, and removing it does not reduce success.

Finding - These results indicate that agents benefit more from information that describes *the failure and how to trigger it* than from the expected behavior. This differs from prior studies of library/framework bugs: Khatib *et al.* found that natural-language S2Rs contribute little to repair success on SWE-bench issues [11, 27], while Bruno *et al.* found that the EB is associated with higher success [11]. A likely explanation is that UI bugs are triggered by sequences of GUI interactions that

are not always easy to infer from the code, making the S2Rs particularly useful to the agent. Concrete cases support this interpretation. For example, for bug #8 in the *TrickyTripper* app, removing the S2Rs leaves  CODEX with only a brief description of the “Create Payment” action.  CODEX then edits an unrelated file rather than addressing the bug. For bug #171 in the *Harmonic* app, removing the OB eliminates the information that the URL in the “Select Text” dialog is truncated.  CODEX still reaches the method changed by the developers, but still fails to address the reported behavior. These examples suggest that S2Rs help the agent reconstruct the triggering interaction, while the OB helps it understand what goes wrong.

Original versus structured reports. The original user reports outperform the structured reports. Across agents, ORIGINAL increases the estimated odds of a plausible patch by a factor of 2.46, whereas STRUCTURED is close to REFERENCE (OR 0.95). This is notable because the original reports are shorter (median 101 versus 169 words) and several omit the EB or S2Rs. The structured reports were created using the quality model of Saha *et al.* [47], which defines the report components we study. Some original reports also contain additional details that are not consistently present across bugs and are therefore not part of the structured format, such as exact values, example inputs, or suggested fixes. **Finding** - the information useful to an agent may extend beyond the conventional report components captured by the structured format.

GUI information and code localization. GUI information has mixed effects (Table 1). Screen descriptions and screenshots have estimates close to one overall, whereas UI hierarchies have the largest positive estimated effect among the GUI components (OR 1.32). The effect is most apparent for  QWEN (Table 2): adding the UI hierarchy in CAUS+SHOT+XML increases its plausible-patch rate from 35% with REFERENCE to 46% (OR 2.00). A **possible explanation** is that UI hierarchies expose resource IDs and component text that agents can search for directly in the source code. In contrast, some configurations containing screen descriptions or screenshots have negative estimated effects for  CODEX and  CLAUDE. One **possible explanation** is that screen descriptions often duplicate information already conveyed by the S2Rs, while some manifesting screens are system dialogs or other UI elements with no direct counterpart in the app code.

Code localization shows the largest positive estimate for  CLAUDE (OR 5.16), with a positive but less certain estimate for  QWEN (OR 1.55) and an estimate near one for  CODEX (OR 0.87). Thus, direct code-level information can substantially change repair success for some agents, but its effect is not uniform. We examine in Section 4.2 whether its stronger effect is related to changes in the agents’ localization behavior and repair effort.

Consistency of the outcomes. Across runs,  CODEX and  CLAUDE are consistent: their three runs give the same result for 86% and 91% of the bug–configuration pairs, compared with only 56% for  QWEN, which explains why many estimates of  QWEN are uncertain (our replication package includes a consistency heatmap [3]). Across configurations, many bugs are fixed within all three attempts or some are not fixed in none.  CODEX fixes 20 bugs in all of its attempts and 5 in none, and  CLAUDE fixes 24 in all attempts and 2 in none. Roughly 50% of the bugs were fixed by  CODEX and  CLAUDE in all three attempts. This is one reason why the effects of the report are moderate when averaged over all bugs. Across agents, the bugs that the agents fix are nested:  CLAUDE fixes every bug that  CODEX fixes, and both fix every bug that  QWEN fixes (in at least half of the attempts). **Finding** - the agents thus agree on which bugs are hard, even though they benefit from different information. Across complexity levels, hard bugs are fixed least often by all agents: CODEX, CLAUDE, and QWEN produce a plausible patch in 52%, 76%, and 15% of their attempts on hard bugs, compared with 67–87%, 89–90%, and 41–47% on easy and medium bugs.

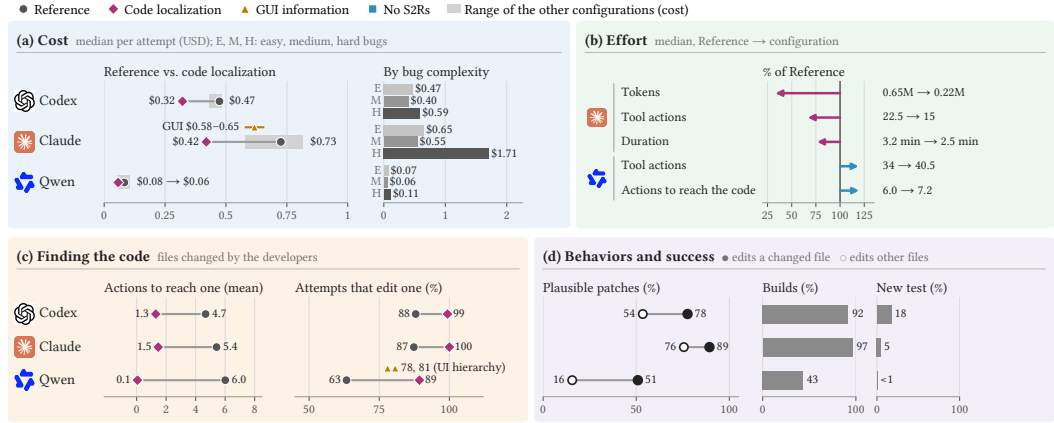


Fig. 3. Behavior of the agents. (a) Median cost per attempt with REFERENCE and code localization (grey band: range of the other configurations) and per bug complexity (E, M, H: easy, medium, hard). (b) Median effort with code localization (CLAUDE) and without the S2Rs (QWEN), relative to REFERENCE. (c) Mean number of actions before the agent first reads a file changed by the developers, and share of attempts that edit such a file. (d) Plausible-patch rate of the attempts that edit a file changed by the developers and of those that edit other files, and share of attempts that build the app and that write a new test.

Summary of Findings for RQ₁

- Bug report information matters, but different agents benefit from different information.
- S2Rs help all agents repair UI bugs, the OB mainly helps CODEX, and the EB adds little.
- Original user reports lead to better results than the structured reports, indicating that useful information extends beyond conventional report components.
- Among GUI information, UI hierarchies are the most promising, while screen descriptions and screenshots have limited or negative effects for some agents.
- Code localization substantially helps CLAUDE, but its benefit does not extend to other agents.
- Agents largely agree on which bugs are difficult, but their run-to-run consistency differs substantially: CODEX and CLAUDE are relatively stable, whereas QWEN is much more variable.

4.2 RQ₂: How Report Content Shapes Agent Behavior and Effort

To understand *how* the report information affects repair success, we analyzed the agents' trajectories. Figure 3 shows how the report changes the effort of the agents, how they find the code to fix, and which of their behaviors are associated with success, and Figure 4 shows where and why their failed attempts fail. The behavior of each agent with every configuration is available in our replication package [3]. We measure the effort of an attempt by its tool actions, duration, tokens, and cost, and how well the agent finds the code to fix by the number of actions before it first reads a file changed by the developers and by whether its patch edits such a file.

Effort and cost. Better information makes the agents cheaper, even when it does not make them more successful. Code localization has the largest effect: it reduces the median cost of an attempt from \$0.47 to \$0.32 for CODEX and from \$0.73 to \$0.42 for CLAUDE (Figure 3(a)), and CLAUDE uses a third of the tokens (0.22 vs. 0.65 million). Both agents also need fewer actions (e.g., 15 vs. 22.5 for CLAUDE) and less time (e.g., 2.5 vs. 3.2 minutes for CLAUDE, Figure 3(b)). GUI information also makes CLAUDE cheaper: four GUI configurations reduce its median cost to between \$0.58 and \$0.65 (Figure 3(a)). Missing information has the opposite effect on QWEN: without the S2Rs, it takes more actions (40.5 vs. 34) and more actions to reach the code to fix (7.2

vs. 6.0, Figure 3(b)). In contrast,  CODEX is the least sensitive to the report, with almost the same cost for all configurations except code localization (Figure 3(a)). The complexity of the bug matters more than the report for  CLAUDE: its median cost is \$1.71 for hard bugs, compared with \$0.65 for easy bugs and \$0.55 for medium bugs (Figure 3(a)). A possible explanation is that  CLAUDE keeps exploring and validating when the fix spans several changes, whereas  CODEX and  QWEN spend about the same effort regardless of the bug.

Finding the code to fix. With code localization, the agents reach a file changed by the developers almost immediately (1.5 actions or fewer, vs. 4.7 to 6.0 with REFERENCE) and edit it in almost every attempt (99% and 100% for  CODEX and  CLAUDE, and 89% vs. 63% for  QWEN, Figure 3(c)). The UI hierarchy helps  QWEN in a similar way: with CAUS+SHOT+XML and CAUS+MAN+SHOT+XML, it edits a file changed by the developers in 78% and 81% of its attempts, compared with 63% with REFERENCE (Figure 3(c)), although it does not reach this file faster. These results suggest that  QWEN uses the resource IDs and texts of the UI hierarchy to search the code, which is consistent with the UI hierarchy being the GUI information that helps it the most in RQ₁.

Behaviors associated with success. Editing a file changed by the developers is the behavior most strongly associated with a plausible patch (Figure 3(d)):  CODEX fixes 78% of the attempts in which it edits such a file, compared with 54% of the others, and  QWEN 51% vs. 16%. A successful build is also associated with success (OR 1.86 across agents, see our replication package [3]). Editing the right file is not always necessary, though:  CLAUDE still fixes 76% of the attempts in which it edits other files, which suggests that the stronger agents find alternative fixes more often. Building the app is a problem mainly for  QWEN, which succeeds in building it in only 43% of its attempts (Figure 3(d)). Surprisingly, attempts in which the agent's last test run passes are *less* likely to succeed (OR 0.47 [3]). The agents rarely test the reported UI scenario:  CODEX writes a new test in 18% of its attempts,  CLAUDE in 5%, and  QWEN almost never (Figure 3(d)). A likely explanation is that the agents run the existing tests or build checks, which pass even when the bug is not fixed and may lead them to stop too early.

Whether the agent edits the right file also explains some of the effects in RQ₁. When we add this behavior as a predictor to the configuration model (results in our replication package [3]), the OR of code localization across agents drops from 1.51 to 0.68, and that of CAUS+SHOT+XML from 1.51 to 1.18. Thus, these configurations help mainly by leading the agent to the right code. For the S2Rs, the opposite happens: the effect of removing them becomes stronger (from 0.65 to 0.55), which suggests that the S2Rs help beyond finding the code, by describing the scenario that the fix has to handle.

Concrete cases support this interpretation. For bug #1146 in the *GPSTest* app, the filter text should disappear when the user stops GNSS. With code localization,  CLAUDE went directly to the method to change and used half as many actions as with REFERENCE (11 vs. 22), but in all three runs it hid the text based on a flag of the screen that does not reflect that GNSS was stopped, and it failed. With REFERENCE,  CLAUDE explored the app further and fixed the bug in two of its three runs. Code localization tells the agent *where* to fix the bug, but not *how*.

Where the agents fail. Most failed attempts change the right code, but the fix is wrong. 48% of them edit a method changed by the developers, and another 17% edit a file changed by the developers, but only 21% edit a different file (see Figure 4(a)). In most of them, the patched app runs but does not behave as expected: 78% of the failed tests end with a failed assertion.  QWEN is the exception, as 22% of its failed attempts produce no patch or no change to the app code, mostly because it stops before producing a patch. Thus, for the stronger agents, finding the code to fix is usually not the problem; writing a fix that produces the expected behavior is.

Why the agent fail. Figure 4(b) reports the root causes of a sample of 338 failed attempts, as labeled by the committee of judges. At least two judges agree on the label of 91% of the attempts, and their agreement is moderate overall (Krippendorff's $\alpha = 0.58$, pairwise Cohen's κ from 0.49 to

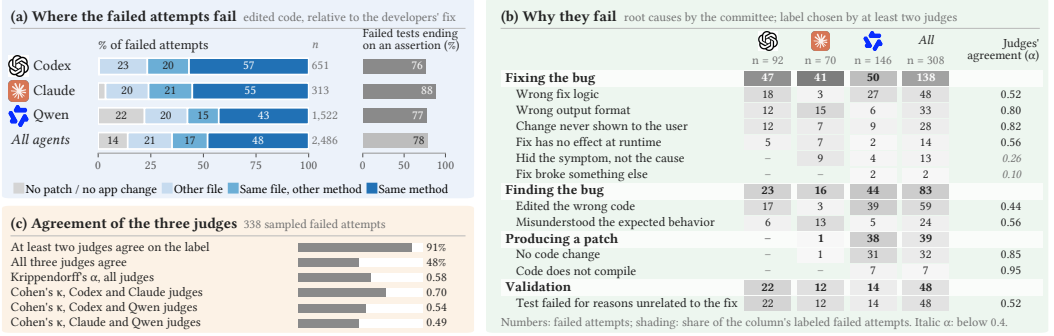


Fig. 4. Failed attempts of the agents. (a) Code edited by the failed attempts relative to the developers' fix, and share of the failed tests that end with a failed assertion. (b) Root causes of the 308 sampled failed attempts whose label is chosen by at least two of the three judges, grouped into four themes, with the agreement of the judges per category (Krippendorff's α). (c) Agreement of the judges on all 338 sampled failed attempts.

0.70, Figure 4(c)), ranging from substantial for clearly observable causes (e.g., $\alpha = 0.95$ for code that does not compile) to low for subtle ones (e.g., $\alpha = 0.26$ for hiding the symptom instead of the cause). The committee confirms the trajectory analysis: most failures happen when *fixing* the bug (45% of the labels), for instance because the fix logic is wrong or the change is never shown to the user, whereas fewer happen when *finding* it (27%).

The root causes also differ across agents. Apart from tests that fail for unrelated reasons, **CODEx** fails mostly because of wrong fix logic or because it edits the wrong code, and **QWEN** often fails before fixing the bug at all: 38 of its failed attempts produce no code change or do not compile. **CLAUDE**, in contrast, fails mostly because its patch does not produce the exact expected behavior: it shows the wrong output format (e.g., a message instead of a system notification) or misunderstands the expected behavior in 28 of its 70 labeled attempts (Figure 4(b)). These results suggest that, for the strongest agent, the main limitation is how precisely the report specifies the expected outcome on the screen. This does not contradict the small effect of the EB in **RQ₁**. Our EBs describe the intended behavior in one general sentence, and only 4 of the 50 state an exact message or value. For example, the EB of bug #2587 says that the app should warn the user that the address is invalid, but not whether the warning is a dialog, a toast, or a message on the screen. Such an EB tells the agent *what* should change but not *how* the result should appear, so removing it costs little. The original reports, which sometimes contain such concrete details, lead to the most fixes. Finally, for 16% of the failures, the judges could not attribute the failure to the patch, because the reproduction test stopped before reaching the patched behavior (e.g., it could not find a UI element).

Summary of Findings for **RQ₂**

- Better information makes the agents cheaper: code localization reduces the med. cost of **CODEx** and **CLAUDE** by about a third and almost half (respectively), and GUI info. also makes **CLAUDE** cheaper.
- Code localization and the UI hierarchy help agents find the code; the S2Rs help beyond that.
- Editing the right file and building the app are associated with success, but the agents rarely test the reported scenario, and passing their own tests can be misleading.
- Most failed attempts change the right code with a wrong fix; **QWEN** often stops without a patch.

5 Implications

Our findings have implications for bug reporters and reporting tools, for the developers of coding agents, and for researchers who study how agents use mobile app bug reports.

Reports should describe how to trigger the bug and what goes wrong. The S2Rs increased the odds of success for all three agents (OR 1.63), and the OB did so especially for  CODEX (OR 3.38, Table 1). The S2Rs also helped beyond leading the agent to the code to fix (Section 4.2). These are the components that end users often omit or write incompletely [12, 68]. Techniques that guide users while they report the steps and the observed behavior [51, 52] or that check whether the S2Rs are complete [12, 35] are thus as useful for agents as for developers. Such techniques could check a report before it is assigned to an agent.

EB should state what the user should see. While the EB added little (Table 1), 28 of the 70 labeled failures of  CLAUDE produced a different outcome than expected: the patch showed the wrong output (e.g., a message instead of a system notification) or misunderstood the expected behavior (Figure 4(b)). As discussed in Section 4.2, our EBs tell the agent *what* should change, but not *how* the result should appear on the screen: only 4 of the 50 state an exact message or value. Even our two researchers agreed least on the EB (65.8%, Section 3.1.4). Report templates should therefore ask for the observable outcome, such as the exact message and the component that shows it. When the outcome is underspecified, agents could ask the reporter instead of guessing [39, 56]. Whether a more precise EB increases success is an open question for future work.

Report improvement techniques should add structure without removing details. The original reports outperformed the structured ones (OR 2.46 vs. 0.95, Table 2), although they are shorter and several lack the EB or S2Rs. Some of them contain exact values, example inputs, or suggested fixes that our structured format did not keep. Techniques that enhance bug reports [47] should therefore keep the reporter's original text next to the structured components rather than replace it.

GUI information is most useful as a UI hierarchy. Among the GUI information, only the UI hierarchy had a clear positive estimate (OR 1.32, Table 1). With CAUS+SHOT+XML, the plausible-patch rate of  QWEN rose from 35% to 46%, and it edited a file changed by the developers in 78% of its attempts instead of 63% (Figure 3(c)). Screen descriptions and screenshots had estimates close to one, and negative ones for some configurations of  CODEX and  CLAUDE. The UI hierarchy is useful because its resource IDs and texts also appear in the code. Reporting tools that already capture GUI data [38, 52] could attach the hierarchies of the causing and manifesting screens at little cost, and compact representations of the hierarchy can reduce the tokens it costs the agent [45]. When only a screenshot is available, techniques that map GUI information to the code [33] could give agents the same benefit.

Agents need to check the reported behavior on the running app. Most failed attempts edited the right code: 48% edited a method changed by the developers, and 78% of the failed tests ended with a failed assertion (Figure 4(a)). At the same time, the agents rarely tested the reported scenario ( CODEX wrote a new test in 18% of its attempts,  CLAUDE in 5%), and attempts whose last test run passed were less likely to succeed (OR 0.47). The agents seem to rely on the build and the existing tests, which say little about the UI behavior. This is similar to the overfitting problem in program repair, where passing the available tests does not mean the bug is fixed [49]. Agent harnesses for mobile apps should let the agent run the app, replay the S2Rs before and after the change, and check the result on the screen. Techniques that reproduce Android bug reports [18, 57, 67] or let LLMs explore an app's UI [46] could serve as such tools, and a reproduction test generated from the report could tell the agent when to stop [26], as shown for SWE-bench issues [40].

Code localization is worth providing even when it does not increase success. Code localization reduced the median cost of an attempt by 32% for  CODEX and 42% for  CLAUDE, and the agents reached a file changed by the developers within 1.5 actions (Figure 3(a, c)). However, it increased success clearly only for  CLAUDE (Table 2), mainly by leading the agent to the right code (OR from 1.51 to 0.68). As bug #1146 shows, it tells the agent where to fix the bug, but not how. This is in line with results on SWE-bench, where agents edit the location changed by the developers

more often than they resolve the issue [60]. When developers or a localization technique [25, 33, 66] can point to the code, adding it to the report is thus a cheap way to reduce the cost of an attempt, and studies of localization for agents should report this saving in addition to the success rate.

6 Threats to Validity

Construct Validity. We measure repair success using *plausible patches*, a standard metric in automated program repair, which requires a patch to build and pass the available tests but does not guarantee semantic correctness [29, 34, 55]. We evaluate every configuration with the same two benchmark tests, one for the reported bug and one for related functionality (Section 3.4). This provides a consistent basis for comparison and can expose some regressions, but does not establish correctness or eliminate test overfitting.

The structured reports and GUI information were manually constructed and may contain errors or reflect our interpretation of the bugs. We followed the quality model of Saha *et al.* [47], reproduced every bug on the emulator, and had a second researcher validate the artifacts (Section 3.1.4). The root causes of the failures are labeled by LLM judges, which can be wrong, especially for subtle causes. We report their agreement per category, since high agreement among models indicates the labels that can be trusted [4]. Some trajectory metrics also rely on classification rules and approximations. We applied the same rules to all agents and release the parsers (see the replication package [3]).

Internal Validity. LLM-based agents are non-deterministic, so a single run may not represent typical behavior. We therefore ran each agent three times for every bug and configuration and analyze run-to-run consistency in Section 4.1.

Because the apps and fixes are public, the models may have encountered their code or fixes during training. This exposure is shared across configurations of the same bug and therefore does not affect comparisons that change only the report. We also disabled network access from the experimental environment, preventing direct retrieval of fixes from external repositories. This does not rule out information available to the models or their providers outside our environment.

We use the same prompt template for all agents and configurations, varying only the report. Because the agents differ in models, tools, harnesses, and logging formats, we compare configurations within each agent and explicitly test differences across agents (Section 3.5).

External Validity. Our study covers 50 real UI-centric bugs from 23 open-source Android apps. Our findings may not generalize to other apps, or platforms. However, the dataset includes multiple bug types and application domains, providing diversity within this scope. Results may also not generalize to other coding agents or LLMs. We selected three agents from different providers, including one based on an open-weight model, but agents and models evolve rapidly. Our replication package enables the study to be repeated with newer agents.

Conclusion Validity. We perform multiple statistical comparisons and control false-positive risk using the Holm correction within each family of comparisons. Some agent-specific estimates are imprecise, as reflected in their wide confidence intervals, so we emphasize effect sizes and intervals rather than statistical significance alone. Finally, analyses relating agent behavior to repair success estimate associations rather than causal effects.

7 Related Work

7.1 Bug Report Quality

Bug report quality has been widely studied from the perspective of human developers. Prior work identifies the Steps to Reproduce (S2Rs), Observed Behavior (OB), and Expected Behavior (EB) as important report components, but also finds that these components are often missing or poorly written [8, 13, 68]. Subsequent work has developed techniques to detect missing information [9,

13, 50], improve S2Rs [12, 35], and help users write better reports through auto-completion [38], chatbots [6], and LLM-based report generation [47]. Rather than improving reports for human developers, we study which report information helps AI coding agents repair UI-centric bugs.

7.2 Bug Reports Studies for Agentic Program Repair

LLM-based agents now resolve issues autonomously [10, 58, 64, 66], and even simpler pipelines that localize the code before repairing it perform competitively [60]. Recent work has examined how issue information affects coding agents in solving bugs on SWE-bench [22, 58, 64]. Meng *et al.* [37] found that issue quality, particularly the presence of line-level locations, is associated with successful resolution, while Aleithan *et al.* [63] found that about one third of resolved SWE-bench issues contain the solution in the issue or its comments. Khatib *et al.* [27] analyzed 27 report features across 87 agents on SWE-bench Verified, and Bruno *et al.* [11] systematically removed information from SWE-bench issues. Both found that agents benefit from operational and code-level information, such as fix locations, suggested fixes, and reproduction scripts, while natural-language S2Rs contribute little to repair success. Other studies examine report refinement from agent trajectories [16], attention to report sections during repair [15], and semantic alignment among reports, tests, and fixes [43].

Our results show that these findings do not fully transfer to UI-centric bugs. On our Android bugs, S2Rs have a positive estimated effect across all three agents, while the EB adds little. Code localization has a large positive effect for  CLAUDE, but not consistently across agents. This difference may stem from the nature of UI bugs: their failures are experienced through GUI interactions, and end-user reports describe what users see and do rather than where the fault lies in the code. Thus, report information that helps agents repair developer-written library and framework issues does not necessarily have the same value for UI bugs.

7.3 GUI Information and Program Repair of UI-centric Apps

Recent work uses visual information for automated issue resolution. SWE-bench Multimodal [64] adds screenshots to issues for JavaScript libraries, while GUIRepair [21] and VisualRepair [62] use multimodal models to reason about screenshots. MM-IssueLoc [65] studies how issue images affect bug localization. Earlier, visual information was used to repair broken web tests [53]. These studies focus on screenshots, specific tasks, or bugs for programs beyond UI-centric software.

Program repair for Android UI bugs is challenging. Mahmud *et al.* [34] found that traditional/non-agentic APR techniques fail on many UI-centric bugs, including bugs involving incorrect or non-existent component identifiers. Other techniques target specific UI bug classes, such as accessibility [36] and compatibility [32]. To the best of our knowledge, our study is the first to examine how different report information affects AI coding agents repairing UI-centric Android bugs.

8 Conclusions and Future Work

We studied 7,650 repair attempts by three APR agents on 50 real bugs, and found that bug report information affects repair success, but its effects depended on both the information and the agent. S2Rs had positive estimated effects across all three agents, the OB had a particularly strong effect for  CODEX, and the EB provided little additional benefit. Original user reports led to more successful repairs than the structured reports, while UI hierarchies were the most promising form of GUI information. Code localization substantially increased the estimated odds of success for  CLAUDE, but not consistently across agents. Report information also affected repair effort and behavior: localization helped agents reach the relevant code faster and reduced the cost of  CODEX and  CLAUDE. Finding the relevant code did not guarantee a successful repair, as many failed attempts reached the relevant code but still produced incorrect behavior. These findings suggest that there is

no single configuration of bug report information that benefits all coding agents. For UI-centric bugs, traditional behavioral information remains useful, and GUI information can help, but its value depends on how it is represented and on the agent.

Our results open several research directions. First, techniques could enrich user reports for agents with the information that helps them most, namely the S2Rs, UI hierarchies, and a concrete expected outcome. Second, agents could reproduce the bug on a device and ask the reporter for missing information, instead of relying on a static report. Third, since hard bugs are fixed least often and cost 🌟 CLAUDE the most, future techniques could estimate the complexity of a bug from its report and adapt the support they provide, e.g., running fault localization [25] to reduce the cost of the agent, or asking the reporter for more information on complex bugs. Finally, our study should be extended to more agents, apps, and platforms, and to reports written by real users for agents.

Data Availability

For verifiability, reproducibility, and facilitating future research, we have made all the study artifacts (e.g., dataset, report configurations, agent trajectories, generated patches, experiment infrastructure, analysis scripts, and documentation) publicly available [3].

References

- [1] 2024. Anglers' Log Bug ID 1445 - <https://github.com/cohenadair/anglers-log/issues/347>.
- [2] 2026. Android UIAutomator- <https://developer.android.com/training/testing/other-components/ui-automator>.
- [3] 2026. Online Replication Package. <https://anonymous.4open.science/r/bug-report-info-apr-replication-package-20C1>. Accessed: 2026-10-02.
- [4] Toufique Ahmed, Premkumar Devanbu, Christoph Treude, and Michael Pradel. 2025. Can LLMs Replace Manual Annotation of Software Engineering Artifacts?. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 526–538.
- [5] Anthropic. 2026. Claude Code. <https://code.claude.com/docs>. Version 2.1.181. Accessed: 2026-09-25.
- [6] Anonymous Authors. 2022. Toward Interactive Bug Reporting for (Android App) End-Users. *Unpublished* (2022).
- [7] Douglas Bates, Martin Mächler, Ben Bolker, and Steve Walker. 2015. Fitting linear mixed-effects models using lme4. *Journal of statistical software* 67 (2015), 1–48.
- [8] Nicolas Bettenburg, Sascha Just, Adrian Schröter, Cathrin Weiss, Rahul Premraj, and Thomas Zimmermann. 2008. What Makes a Good Bug Report?. In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Atlanta, Georgia). 308–318. doi:10.1145/1453101.1453146
- [9] Lili Bo, Wangjie Ji, Xiaobing Sun, Ting Zhang, Xiaoxue Wu, and Ying Wei. 2024. ChatBR: Automated assessment and improvement of bug report quality using ChatGPT. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA). 1472–1483. doi:10.1145/3691620.3695518
- [10] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2188–2200.
- [11] Vincenzo Luigi Bruno, Alessandro Giagnorio, Daniele Bifulco, Leon Wienges, Massimiliano Di Penta, and Gabriele Bavota. 2026. Writing Bug Reports for Software Repair Agents: What Information Matters Most? *arXiv preprint arXiv:2607.09553* (2026).
- [12] Oscar Chaparro, Carlos Bernal-Cárdenas, Jing Lu, Kevin Moran, Andrian Marcus, Massimiliano Di Penta, Denys Poshyvanyk, and Vincent Ng. 2019. Assessing the Quality of the Steps to Reproduce in Bug Reports. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Tallinn, Estonia). 86–96. doi:10.1145/3338906.3338947
- [13] Oscar Chaparro, Jing Lu, Fiorella Zampetti, Laura Moreno, Massimiliano Di Penta, Andrian Marcus, Gabriele Bavota, and Vincent Ng. 2017. Detecting Missing Information in Bug Descriptions. In *Proceedings of the 11th Joint Meeting on the Foundations of Software Engineering (ESEC/FSE'17)*. 396–407. doi:10.1145/3106237.3106285
- [14] Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46. doi:10.1177/001316446002000104
- [15] Ramtin Ehsani, Irene Manotas, Saurabh Pujar, Luca Buratti, and Preetha Chatterjee. 2026. How Do LLMs Read Bug Reports? An Empirical Study of Attention in LLMs for Automated Program Repair. *arXiv preprint arXiv:2607.25873* (2026).
- [16] SM Fahim, Md Nakhla Rafi, Md Ahasanuzzaman, Zeyang Ma, Dong Jae Kim, Shaowei Wang, et al. 2026. Bug Report Specification Refinement with Trajectory Guidance for Automated Program Repair. *arXiv preprint arXiv:2607.07882*

- (2026).
- [17] Sidong Feng and Chunyang Chen. 2022. GIFdroid: Automated Replay of Visual Bug Reports for Android Apps. In *Proceedings of the 44th International Conference on Software Engineering (Pittsburgh, Pennsylvania) (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 1045–1057. doi:10.1145/3510003.3510048
 - [18] Sidong Feng and Chunyang Chen. 2024. Prompting Is All You Need: Automated Android Bug Replay with Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (Lisbon, Portugal)*. doi:10.1145/3597503.3608137
 - [19] Walter D Fisher. 1958. On grouping for maximum homogeneity. *Journal of the American statistical Association* 53, 284 (1958), 789–798.
 - [20] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.
 - [21] Kai Huang, Jian Zhang, Xiaofei Xie, and Chunyang Chen. 2025. Seeing is fixing: Cross-modal reasoning with multimodal llms for visual software issue repair. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1156–1168.
 - [22] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swebench: Can language models resolve real-world github issues?. In *International Conference on Learning Representations*, Vol. 2024. 54107–54157.
 - [23] Jack Johnson, Junayed Mahmud, Oscar Chaparro, Kevin Moran, and Mattia Fazzini. 2025. Generating Failure-Based Oracles to Support Testing of Reported Bugs in Android Apps. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2196–2208.
 - [24] Jack Johnson, Junayed Mahmud, Tyler Wendland, Kevin Moran, Julia Rubin, and Mattia Fazzini. 2022. An empirical investigation into the reproduction of bug reports for android apps. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 321–322. doi:10.1109/SANER53432.2022.00048
 - [25] Sungmin Kang, Gabin An, and Shin Yoo. 2024. A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1424–1446.
 - [26] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2312–2323. doi:10.1109/ICSE48619.2023.00194
 - [27] Lara Khatib, Noble Saji Mathews, Meiyappan Nagappan, Pengyu Nie, and Thomas Zimmermann. 2026. What Makes a Good Bug Report for an AI Agent? *arXiv preprint arXiv:2607.07593* (2026).
 - [28] Klaus Krippendorff. 2018. *Content analysis: An introduction to its methodology*. Sage publications.
 - [29] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated program repair. *Commun. ACM* 62, 12 (2019), 56–65.
 - [30] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2026. Large Language Model-Based Agents for Software Engineering: A Survey. *ACM Transactions on Software Engineering and Methodology* (2026).
 - [31] Zhe Liu, Chunyang Chen, Junjie Wang, Yuekai Huang, Jun Hu, and Qing Wang. 2021. Owl eyes: spotting UI display issues via visual understanding. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (Virtual Event, Australia) (ASE '20)*. Association for Computing Machinery, New York, NY, USA, 398–409. doi:10.1145/3324884.3416547
 - [32] Zhijie Liu, Yutian Tang, Meiyun Li, Xin Jin, Yunfei Long, Liang Feng Zhang, and Xiapu Luo. 2024. LLM-CompDroid: Repairing Configuration Compatibility Bugs in Android Apps with Pre-trained Large Language Models. arXiv:2402.15078 [cs.SE] <https://arxiv.org/abs/2402.15078>
 - [33] Junayed Mahmud, Nadeeshan De Silva, Safwat Ali Khan, Seyed Hooman Mostafavi, S M Hasan Mansur, Oscar Chaparro, Andrian (Andi) Marcus, and Kevin Moran. 2024. On Using GUI Interaction Data to Improve Text Retrieval-based Bug Localization. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (Lisbon, Portugal)*. doi:10.1145/3597503.3608139
 - [34] Junayed Mahmud, Sparsh Pandey, Nadeeshan De Silva, Atish Kumar Dipongkor, JingJing Wu, Oscar Chaparro, Mattia Fazzini, and Kevin Moran. 2026. Automated Program Repair for UI-centric Android Bugs: How Far are We?. In *Proceedings of the 35th ACM International Symposium on Software Testing and Analysis*. To appear. doi:10.1145/3832272
 - [35] Junayed Mahmud, Antu Saha, Oscar Chaparro, Kevin Moran, and Andrian Marcus. 2025. Combining Language and App Ui Analysis for the Automated Assessment of Bug Reproduction Steps. In *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*. 1–12. doi:10.1109/ICPC66645.2025.00010
 - [36] Forough Mehralian, Navid Salehnamadi, and Sam Malek. 2021. Data-driven accessibility repair revisited: on the effectiveness of generating labels for icons in Android apps. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Athens, Greece) (ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 107–118. doi:10.1145/3468264.3468604

- [37] Xiangxin Meng, Zexiong Ma, Pengfei Gao, and Chao Peng. 2024. An empirical study on llm-based agents for automated bug fixing. *arXiv preprint arXiv:2411.10213* (2024).
- [38] Kevin Moran, Mario Linares-Vásquez, Carlos Bernal-Cárdenas, and Denys Poshyvanyk. 2015. Auto-completing bug reports for Android applications. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (Bergamo, Italy) (ESEC/FSE 2015)*. Association for Computing Machinery, New York, NY, USA, 673–686. doi:10.1145/2786805.2786857
- [39] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, ChenXue Wang, Shichao Liu, and Qing Wang. 2024. Clarifygpt: A framework for enhancing llm-based code generation via requirements clarification. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2332–2354.
- [40] Niels Mündler, Mark N Müller, Jingxuan He, and Martin Vechev. 2024. Swt-bench: Testing and validating real-world bug-fixes with code agents. *Advances in Neural Information Processing Systems* 37 (2024), 81857–81887.
- [41] Noor Nashid, Daniel Ding, Keheliya Gallaba, Ahmed E Hassan, and Ali Mesbah. 2025. Characterizing Multi-Hunk Patches: Divergence, Proximity, and LLM Repair Challenges. CoRR abs/2506.04418 (2025). *arXiv preprint arXiv:2506.04418* (2025).
- [42] OpenAI. 2026. Codex CLI. <https://github.com/openai/codex>. Version 0.138.0. Accessed: 2026-09-25.
- [43] Wendkúuni C Ouédraogo, Yinghua Li, Xueqi Dang, Paweł Borsukiewicz, Liang Xiao, Lingfeng Bao, Anil Koyuncu, Jacques Klein, David Lo, and Tegawendé F Bissyandé. 2026. Semantic Drift in Bug Resolution: How Behavioral Signals Propagate from Reports to Tests and Patches. *arXiv preprint arXiv:2607.18550* (2026).
- [44] Qwen Team. 2026. Qwen Code. <https://github.com/QwenLM/qwen-code>. Version 0.24.2. Accessed: 2026-09-25.
- [45] Dezhi Ran, Zhi Gong, Yuzhe Guo, Mengzhou Wu, Yuan Cao, Haochuan Lu, Hengyu Zhang, Xia Zeng, Gang Cao, Liangchao Yao, Yuetang Deng, Wei Yang, and Tao Xie. 2025. From User Interface to Agent Interface: Efficiency Optimization of UI Representations for LLM Agents. *arXiv preprint arXiv:2512.13438* (2025).
- [46] Dezhi Ran, Hao Wang, Zihe Song, Mengzhou Wu, Yuan Cao, Ying Zhang, Wei Yang, and Tao Xie. 2024. Guardian: A Runtime Framework for LLM-Based UI Exploration. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 958–970.
- [47] Antu Saha, Atish Kumar Dipongkor, Sam Bennett, Kevin Moran, Andrian Marcus, and Oscar Chaparro. 2026. Automated Generation of High-Quality Bug Reports for Android Applications. *arXiv preprint arXiv:2604.01148* (2026).
- [48] Antu Saha, Yang Song, Junayed Mahmud, Ying Zhou, Kevin Moran, and Oscar Chaparro. 2024. Toward the Automated Localization of Buggy Mobile App UIs from Bug Descriptions. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1249–1261. doi:10.1145/3650212.3680357
- [49] Edward K Smith, Earl T Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the cure worse than the disease? overfitting in automated program repair. In *Proceedings of the 2015 10th joint meeting on foundations of software engineering*. 532–543.
- [50] Yang Song and Oscar Chaparro. 2020. Bee: A tool for structuring and analyzing bug reports. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1551–1555. doi:10.1145/3368089.3417928
- [51] Yang Song, Junayed Mahmud, Nadeeshan De Silva, Ying Zhou, Oscar Chaparro, Kevin Moran, Andrian Marcus, and Denys Poshyvanyk. 2023. BURT: A Chatbot for Interactive Bug Reporting. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering*. 170–174. doi:10.1109/ICSE-Companion58688.2023.00048
- [52] Yang Song, Junayed Mahmud, Ying Zhou, Oscar Chaparro, Kevin Moran, Andrian Marcus, and Denys Poshyvanyk. 2022. Toward interactive bug reporting for (android app) end-users. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 344–356. doi:10.1145/3540250.3549131
- [53] Andrea Stocco, Rahulkrishna Yandrapally, and Ali Mesbah. 2018. Visual web test repair. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 503–514.
- [54] Ting Su, Jue Wang, and Zhendong Su. 2021. Benchmarking automated GUI testing for Android against real-world bugs. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 119–130.
- [55] Haoye Tian, Kui Liu, Abdoul Kader Kaboré, Anil Koyuncu, Li Li, Jacques Klein, and Tegawendé F Bissyandé. 2020. Evaluating representation learning of code changes for predicting patch correctness in program repair. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 981–992.
- [56] Sanidhya Vijayvargiya, Xuhui Zhou, Akhila Yerukola, Maarten Sap, and Graham Neubig. 2025. Interactive agents to overcome ambiguity in software engineering. *arXiv preprint arXiv:2502.13069* (2025).
- [57] Dingbang Wang, Yu Zhao, Sidong Feng, Zhaoxu Zhang, William G. J. Halfond, Chunyang Chen, Xiaoxia Sun, Jiangfan Shi, and Tingting Yu. 2024. Feedback-Driven Automated Whole Bug Report Reproduction for Android Apps. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria). 1048–1060. doi:10.1145/3650212.3680341

- [58] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2025. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, Vol. 2025. 65882–65919.
- [59] Tyler Wendland, Jingyang Sun, Junayed Mahmud, Syeda Mansur, Steven Huang, Kevin Moran, Julia Rubin, and Mattia Fazzini. 2021. Andror2: A Dataset of Manually-Reproduced Bug Reports for Android apps. *Proceedings of the 18th IEEE/ACM International Conference on Mining Software Repositories (MSR'21)* (2021), 600–604.
- [60] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 801–824.
- [61] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using chatgpt. In *Proceedings of the 33rd ACM SIGSOFT international symposium on software testing and analysis*. 819–831.
- [62] Jingyu Xiao, Zhongyi Zhang, Haoran Hou, Yuxuan Wan, Yuan Jiang, Yintong Huo, and Michael R Lyu. 2026. VisualRepair: Dynamic Tool Calling and Region Focusing for Visual Software Issue Repair. *arXiv preprint arXiv:2607.14075* (2026).
- [63] Haoran Xue, Reem Aleithan, Nafid Enan, Gias Uddin, and Song Wang. 2026. SWE-Bench+: Enhanced LLM Coding Benchmark. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software*. 332–339.
- [64] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.
- [65] Shaoxiong Zhan, Shi Hu, Boyu Feng, Hai Lin, Andrew Gong, Zhengda Zhou, Jiaying Zhou, Yunyun Hou, Hao Su, and Hai-Tao Zheng. 2026. MM-IssueLoc: A Controlled Benchmark for Evaluating Visual Evidence in Multimodal Repository-Level Issue Localization. *arXiv preprint arXiv:2607.15205* (2026).
- [66] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1592–1604.
- [67] Yu Zhao, Tingting Yu, Ting Su, Yang Liu, Wei Zheng, Jingzhi Zhang, and William G.J. Halfond. 2019. ReCDroid: Automatically Reproducing Android Application Crashes from Bug Reports. In *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering (ICSE)*. 128–139. doi:10.1109/ICSE.2019.00030
- [68] Thomas Zimmermann, Rahul Premraj, Nicolas Bettenburg, Sascha Just, Adrian Schroter, and Cathrin Weiss. 2010. What Makes a Good Bug Report? *IEEE Transactions on Software Engineering* 36, 5 (2010), 618–643. doi:10.1109/TSE.2010.63

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009